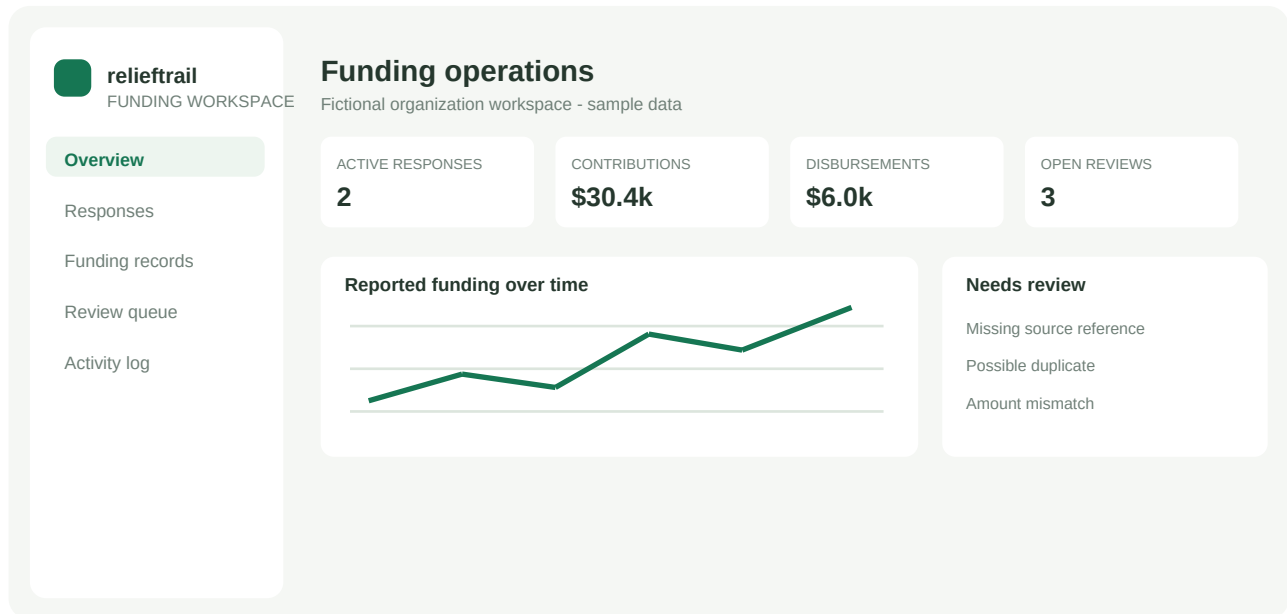


ReliefTrail Funding Workspace

An organization-scoped web application for organizing reported relief-response funding, reviewing mismatches, and keeping a traceable action history.



Illustrative interface preview based on the frontend; live database-connected capture is pending local Docker availability.

Project stage

Working portfolio demo: React + TypeScript frontend, FastAPI backend, and PostgreSQL schema with fictional sample records. Local Docker was unavailable during report preparation, so this document uses an illustrative interface preview rather than a live database screenshot.

Safety note

All data and demo credentials are fictional. The app does not accept donations, move money, or verify aid delivery. This project is not approved for real organizational data or production use.

1. Introduction

ReliefTrail Funding Workspace is a database-backed portfolio application for a fictional nonprofit team. It brings relief response context and reported contribution/disbursement records into the same workspace. Staff can import a partner CSV, inspect human review flags, and view an action history.

2. Problem statement

Funding details may arrive in separate spreadsheets and records. This makes it harder to see which response a line belongs to, spot missing references, and explain what a reviewer changed. The project demonstrates a small internal workflow that links the records and preserves the source context. It is a learning scenario, not a verified statement about a particular organization's operational pain.

3. Objectives

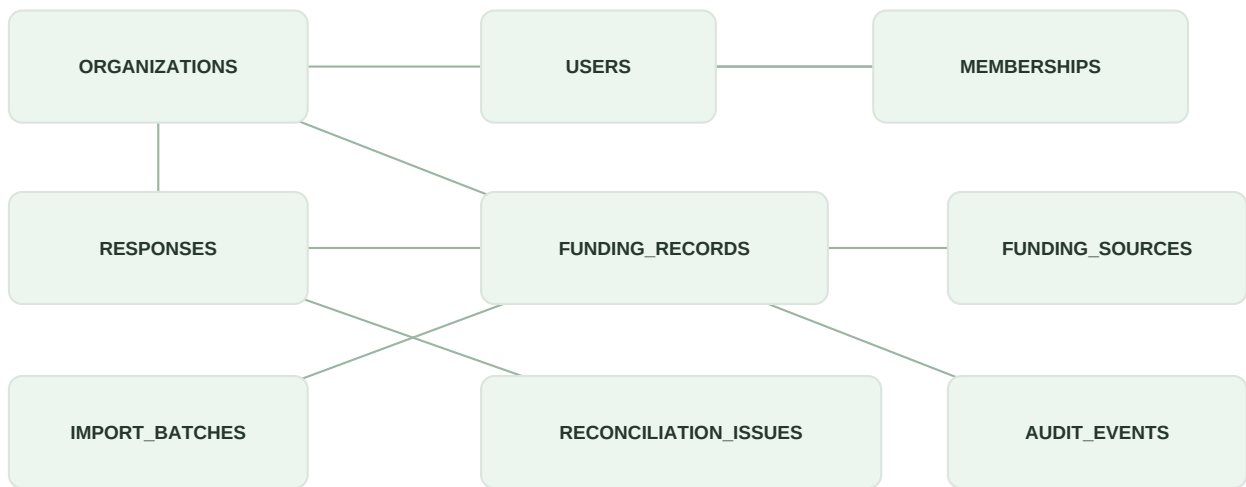
- Design normalized relational entities with primary keys, foreign keys, constraints, indexes, and organization membership.
- Build a friendly interface for response create/read/update/delete, CSV import/export, review flags, and activity history.
- Use SQL joins, aggregates, a reporting view, transaction-backed writes, and a trigger.
- Demonstrate authentication and role-aware organization access checks in a controlled fictional demo.

4. Technologies

Layer	Technology	Purpose
Frontend	React, TypeScript, Vite	Responsive workspace interface
Backend	Python, FastAPI, Pydantic	JSON API, validation, authorization
Database	PostgreSQL 16	Relational storage, views, trigger, transactions
Local run	Docker Compose, pnpm	Repeatable demo setup

5. ER diagram and database schema

The design separates organization, identity, response, funding, review, and audit facts. Foreign keys show how those facts connect.



Each line represents a foreign-key relationship; membership connects users to organizations.

Table	Key columns and purpose
organizations	id PK; organization name and type
users / organization_memberships	user_id + organization_id composite PK; role-based membership
responses	id PK; organization_id FK; unique response code; status, dates, summary
funding_sources / import_batches	Source identity and organization-scoped CSV import summaries
funding_records	response_id/source_id/import_batch_id FKs; type, amount, currency, date, references
reconciliation_issues	Response FK and optional funding-record FK; issue state
audit_events	Organization-scoped action and actor history; append-only trigger

Normalization

Related facts are stored once and connected with keys: response rows refer to organizations; funding rows refer to responses and sources. The reporting view calculates totals from funding rows instead of copying totals into response rows. This reduces duplicate data and update anomalies, following a practical third-normal-form style.

6. SQL queries and implemented features

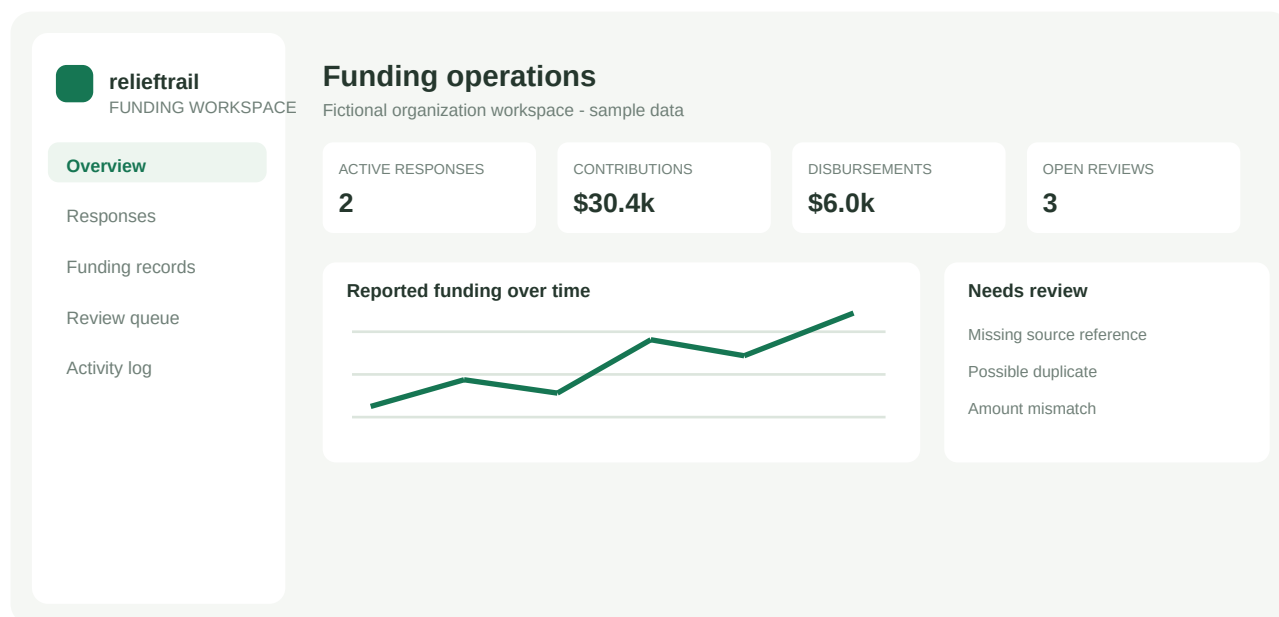
- Parameterized SELECT queries join responses, organizations, funding sources, and review issues.
- Aggregate queries calculate contribution, disbursement, response, and issue summaries within the user's organization memberships.
- The response_funding_summary view derives response totals from funding records.
- Foreign keys, uniqueness rules, and CHECK constraints protect allowed statuses, roles, currency, positive amounts, and response codes.
- CSV validation completes before database writes; accepted rows and the import batch/audit row commit in one transaction. Duplicate source references are skipped.
- An append-only PostgreSQL trigger rejects UPDATE and DELETE on audit_events.
- A signed, expiring bearer token identifies a user. The API scopes reads to organization memberships and checks roles for response changes, CSV imports, and issue resolution.
- Only unused draft responses can be deleted; a funding or review history blocks deletion. The database trigger rejects changes to audit events.

Example relationship query

```
SELECT r.code, o.name AS organization, f.record_type, f.amount, s.name AS source
FROM funding_records f
JOIN responses r ON r.id = f.response_id
JOIN organizations o ON o.id = r.organization_id
JOIN funding_sources s ON s.id = f.source_id;
```

7. Interface preview

The preview summarizes the dashboard layout: a left workspace menu, response and funding summary cards, a chart, and a review panel. The app also includes dedicated response, funding, review queue, and activity screens. This is an illustrative rendering based on the implemented frontend. The database-backed UI has not yet been captured running in a local browser because Docker is not installed on the build host.



Illustrative interface preview based on the frontend; live database-connected capture is pending local Docker availability.

8. Conclusion

This mini project demonstrates an end-to-end database application: a typed frontend calls an API, the API validates requests and enforces organization membership, and PostgreSQL stores related records with constraints, a view, a trigger, and transactional workflows. It provides a practical base for learning schema design, SQL, joins, data validation, authorization, and audit concepts.

9. Future scope

- Add invitation, role-administration, password reset, MFA, and login throttling; adopt established auth libraries and production secrets.
- Add automated schema migrations, authorization-focused verification, deployment monitoring, tested backups, and an independent security review.
- Improve reconciliation rules, spreadsheet column mapping, and correction workflow.
- Explore human-reviewed AI suggestions for mapping columns or drafting reports with source-row citations.
- Connect the earlier contract prototype as a read-only event source after its provenance and integrity are verified.
- Interview NGO grants and finance staff before making adoption or commercial claims.

Repository and setup

Source code, SQL schema, seed data, query examples, and local instructions are in the ReliefTrail Funding Workspace GitHub repository. The initial setup uses Docker Compose; API docs are available at /docs when running locally.